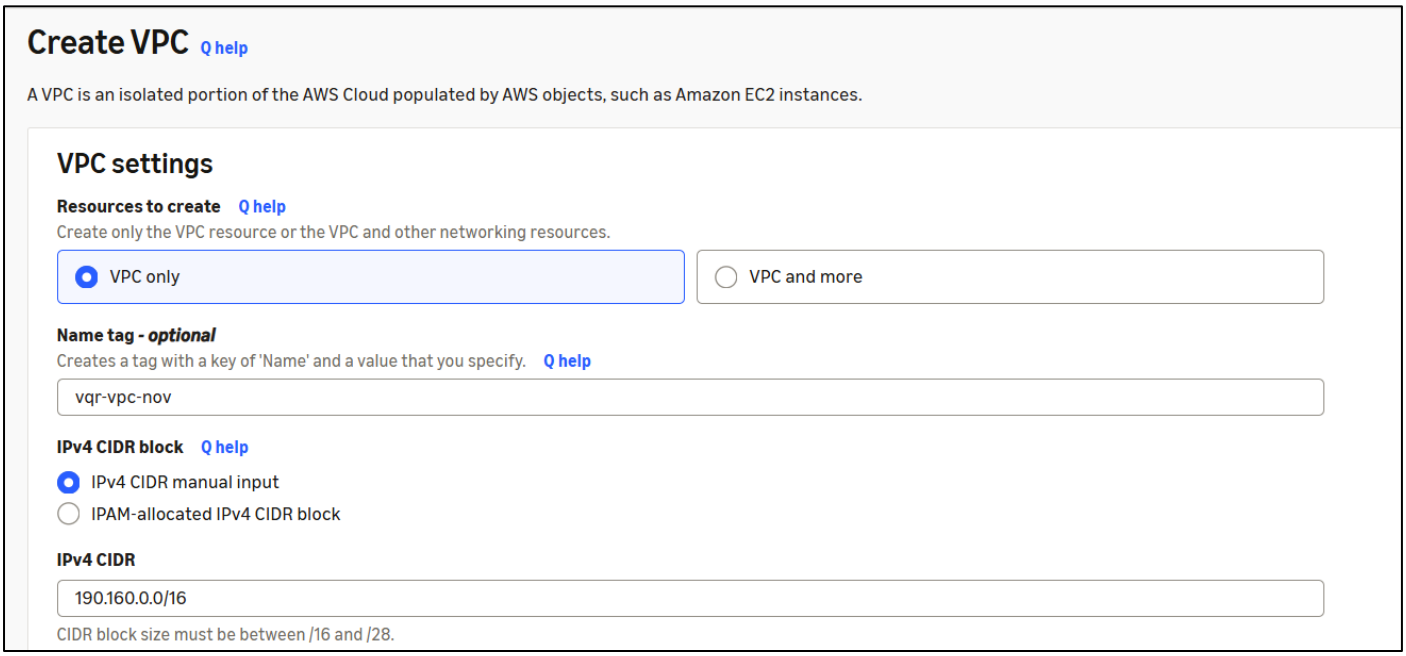
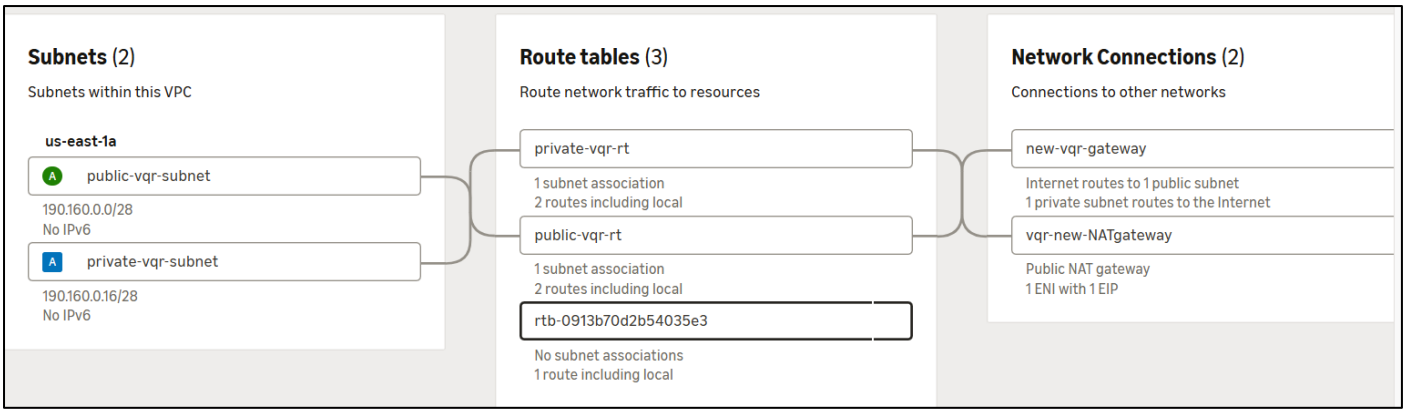
Auto Scaling Group

1. Create one VPC in N. Virginia region.

Click **Create VPC** → Select **VPC only.** → Enter VPC name: My-VPC. → Enter IPv4 CIDR block: 10.0.0.0/16. → Select **No IPv6 CIDR block.** → Select **Default** tenancy. → Click **Create VPC.**
Verify that the VPC is successfully created.

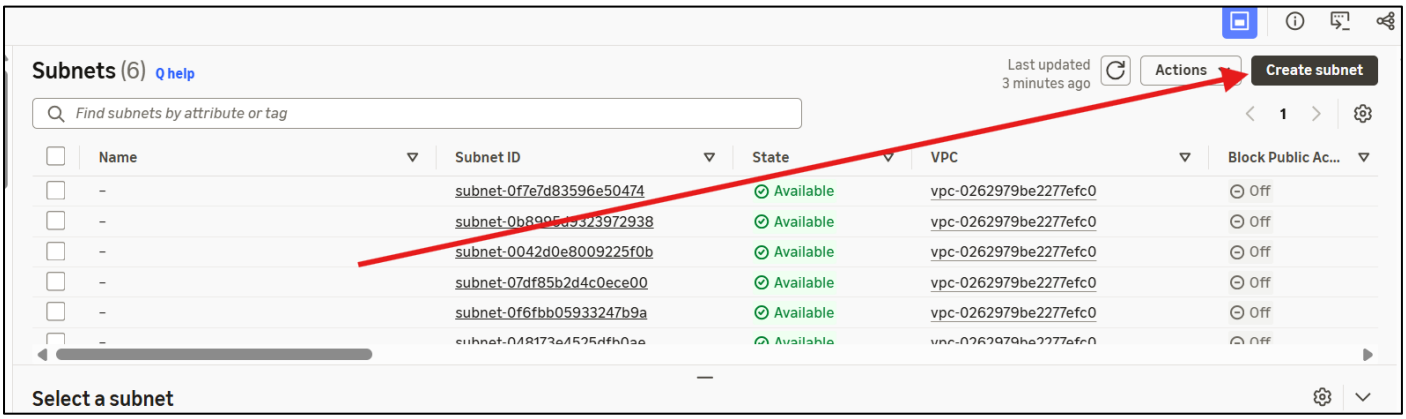


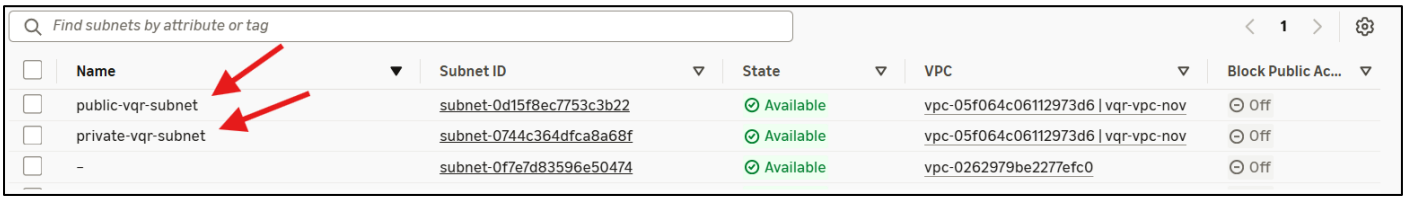


2. Create two subnets: one public subnet and one private subnet.

Click **Create subnet.** → Select the VPC created earlier (My-VPC). **Create Public Subnet** → Enter subnet name: Public-Subnet. → Select Availability Zone (e.g., us-east-1a). → Enter IPv4 subnet CIDR block: 10.0.1.0/24.→ Click **Create subnet**.

Create Private Subnet → Click **Create subnet** again. → Select the same VPC (My-VPC). → Enter subnet name: Private-Subnet. → Select Availability Zone (e.g., us-east-1a). → Enter IPv4 subnet CIDR block: 10.0.2.0/24.

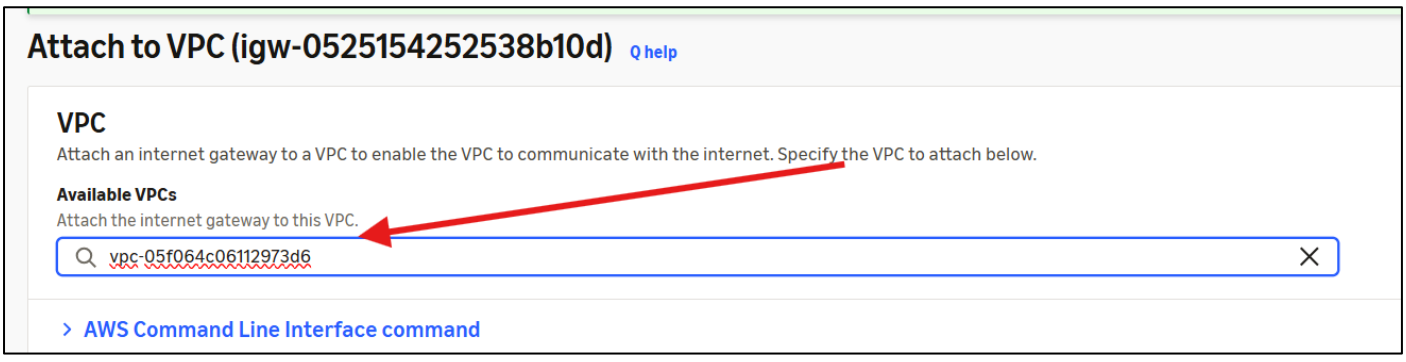




3. Attach an IGW to the VPC.

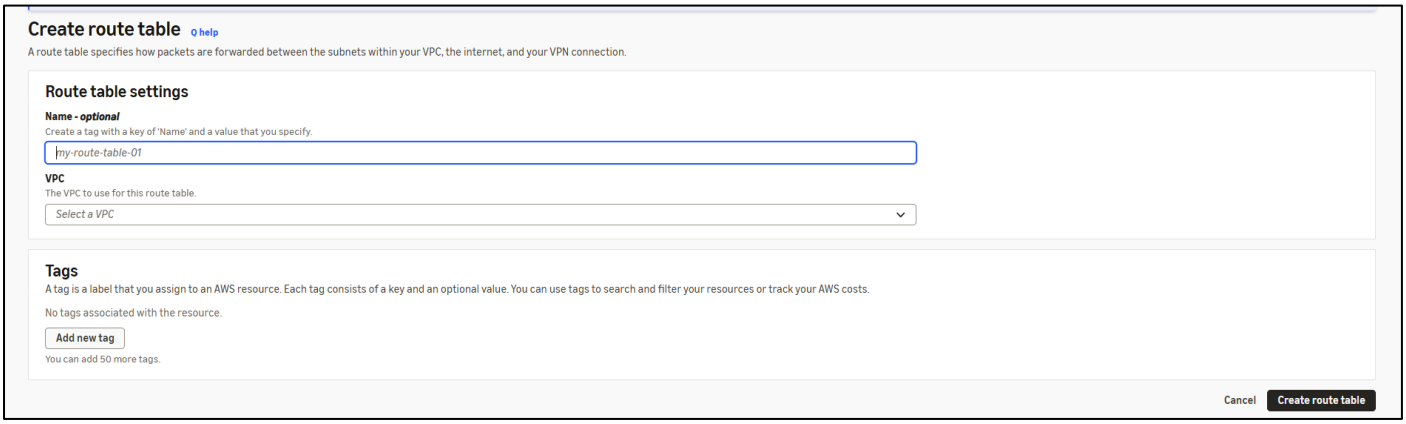
Open the **AWS VPC Dashboard.** → Select **Internet Gateways** from the left navigation pane. → Click **Create internet gateway.** → Enter the name: __ → Click **Create internet gateway.** → Select the newly created Internet Gateway (___). → Click **Actions.** → Select **Attach to a VPC.** → Choose your VPC (Created VPC). → Click **Attach internet gateway.**

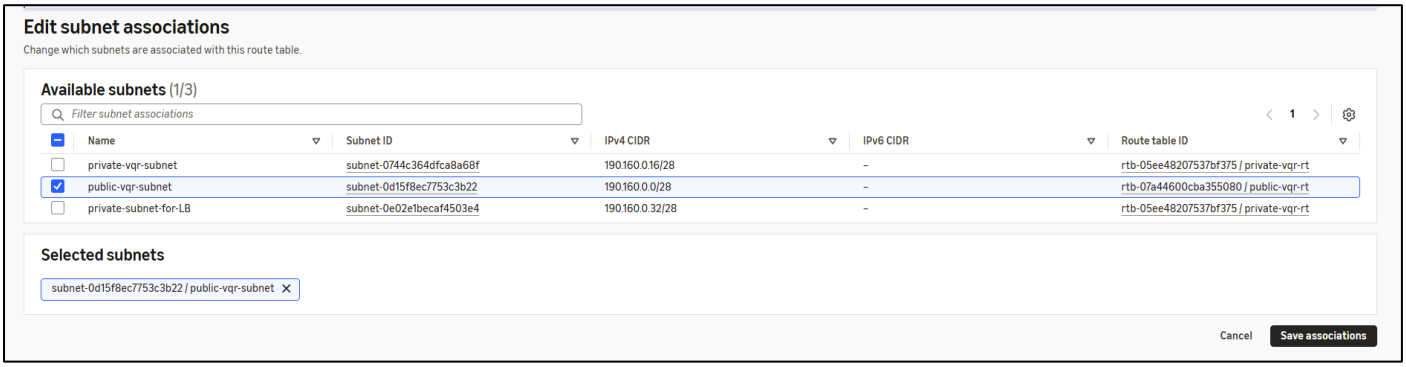


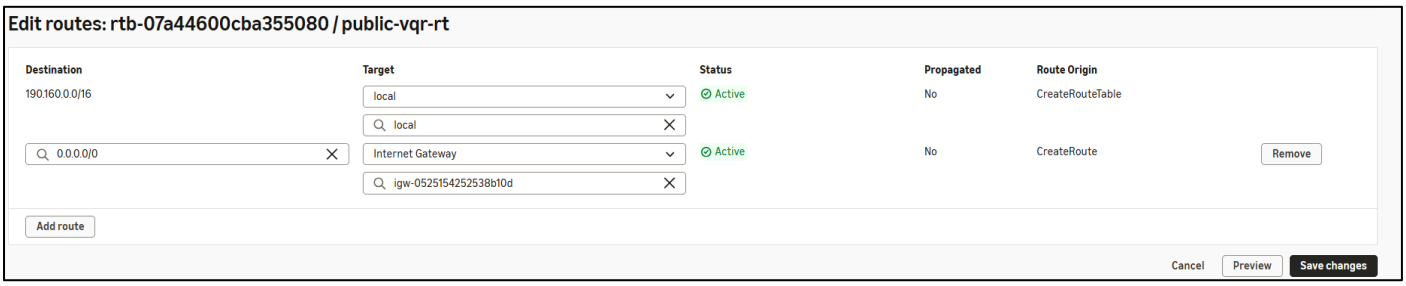


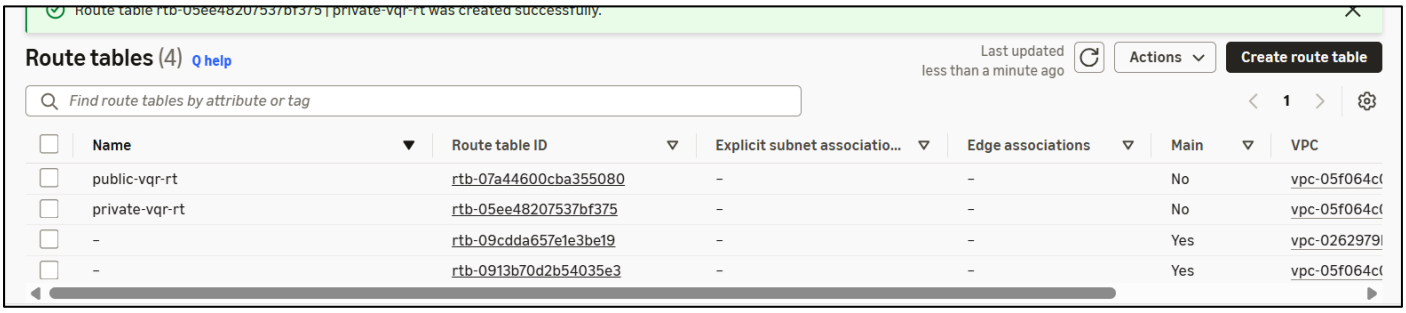
4. Create one public route table (RT) and one private route table.

Select **Route Tables** from the left navigation pane.
Create Public Route Table → Click **Create route table.** → Enter name: Public-RT. → Select your VPC (My-VPC). → Click **Create route table.** → Select Public-RT.
Go to the **Routes** tab and click **Edit routes.** → Click **Add route.** → Enter destination: 0.0.0.0/0.
→ Select target: **Internet Gateway (My-IGW).** → Click **Save changes.** → Go to the **Subnet associations** tab.→ Click **Edit subnet associations.**→ Select Public-Subnet. → Click **Save associations.**
Create Private Route Table → Click **Create route table.** → Enter name: Private-RT. → Select your VPC (My-VPC). → Click **Create route table.** → Select Private-RT. → Go to the **Subnet associations** tab. → Click **Edit subnet associations.** → Select Private-Subnet.→ Click **Save associations.**







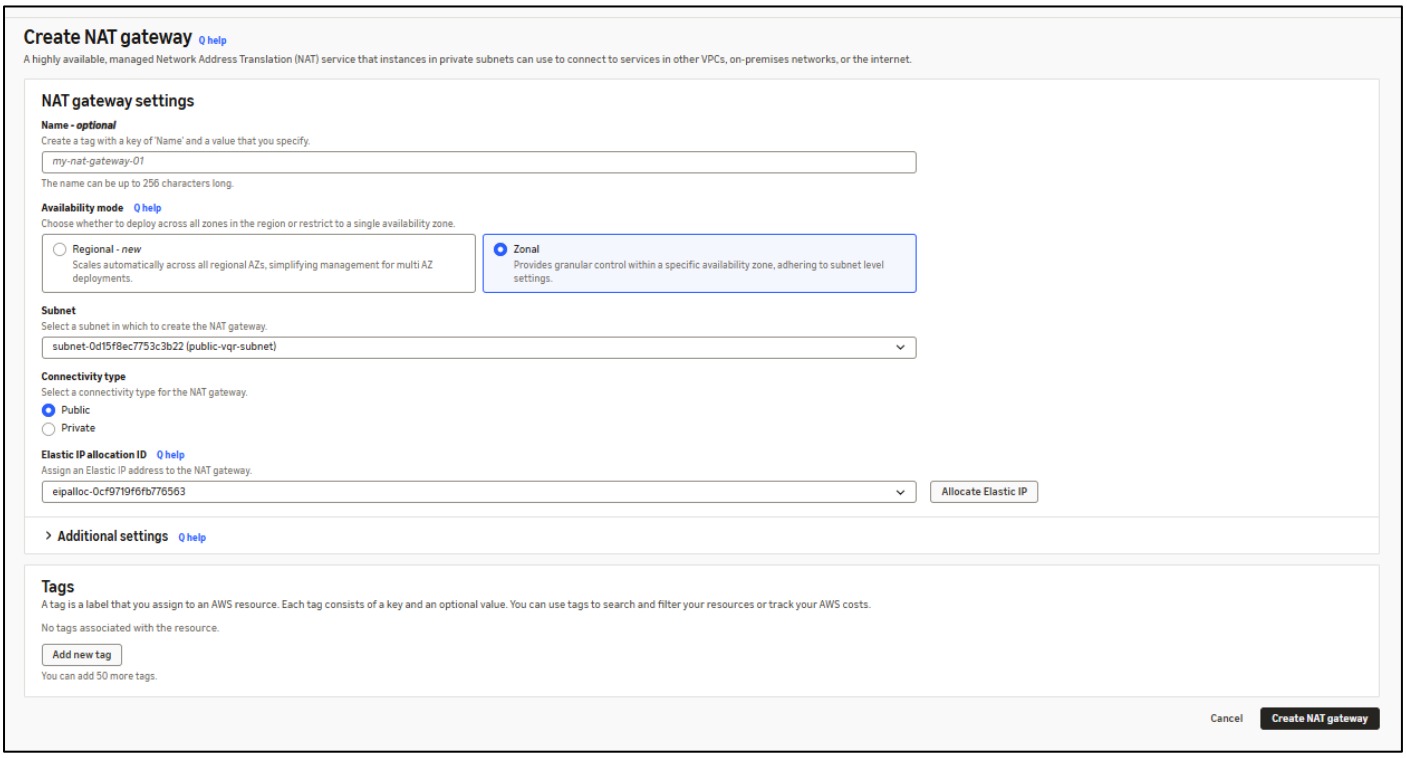


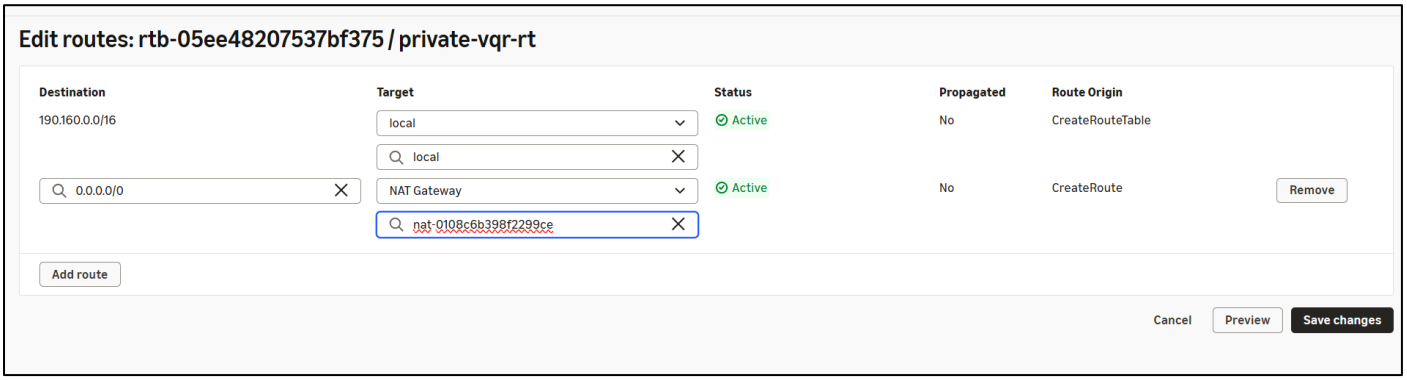
5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

Select **NAT Gateways** from the left navigation pane. → Click **Create NAT gateway**. → Enter name: My-NAT-Gateway. → Select your VPC (My-VPC). → Select subnet: Public-Subnet. → Under Connectivity type, select **Public**. → Under Elastic IP allocation, click **Allocate Elastic IP**. → Click **Create NAT gateway**. → Wait until the NAT Gateway status changes to **Available**.

Attach NAT Gateway to Private Subnet

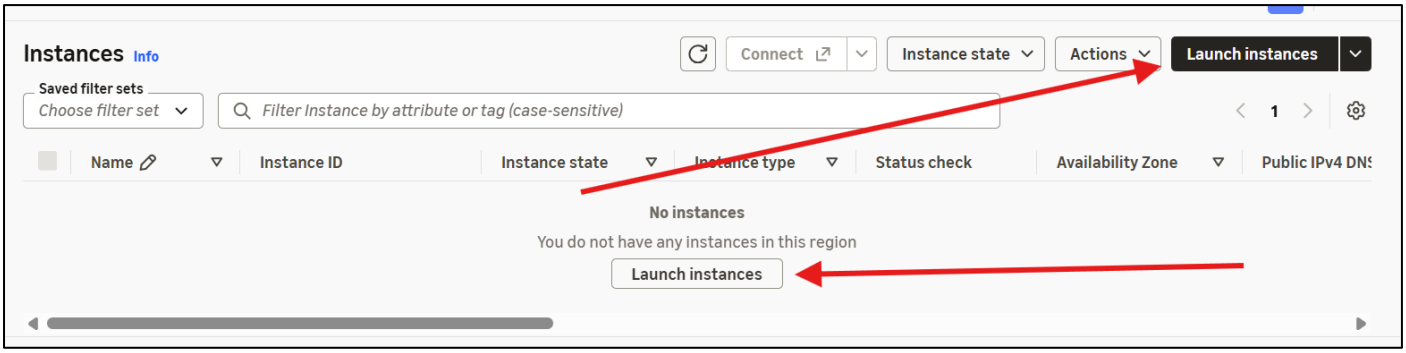
→ Go to Route Tables. → Select Private-RT. → Open the Routes tab. → Click Edit routes. → Click Add route. → Enter destination: 0.0.0.0/0. → Select target: NAT Gateway. → Choose My-NAT-Gateway. → Click Save changes.

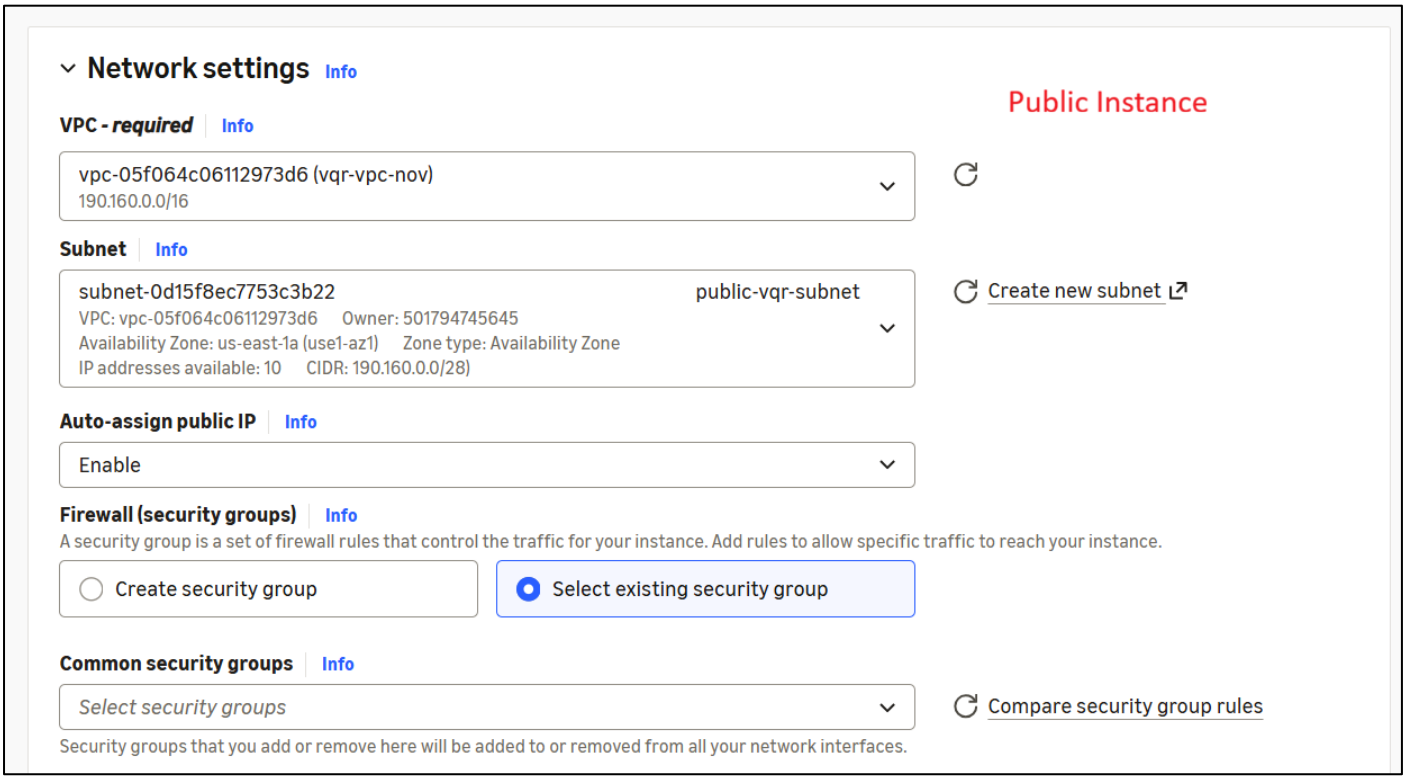




6. Create two instances, one in the public subnet and one in the private subnet.

Open the EC2 Dashboard.
→ Ensure the region is **US East (N. Virginia) – us-east-1.** → **Create Public EC2 Instance** → Click **Instances** → **Launch instances.** → Enter name: Public-Instance. → Select an Amazon Machine Image (AMI), such as **Amazon Linux 2023.** → Select an instance type: t2.micro (if available). → Create or select an existing key pair. → Under **Network settings,** click **Edit.** → Select VPC: My-VPC.→ Select subnet: Public-Subnet. → Enable **Auto-assign public IP.** → select existing security group allowing SSH (port 22) from your IP address.→ Click **Launch instance.**
Create Private EC2 Instance → Click **Launch instances** again. → Enter name: Private-Instance. → Select the same AMI: **Amazon Linux 2023.** → Select instance type: t2.micro (if available). → Select the same key pair. → Under **Network settings,** click **Edit.** → Select VPC: My-VPC. → Select subnet: Private-Subnet.
→ Disable **Auto-assign public IP.** → Select existing security group allowing SSH (port 22) from the public instance's security group. → Click **Launch instance.**





Network settings Info

Private Instance

VPC - *required* Info

vpc-05f064c06112973d6 (vqr-vpc-nov)
190.160.0.0/16

Subnet Info

subnet-0744c364dfca8a68f private-vqr-subnet
VPC: vpc-05f064c06112973d6 Owner: 501794745645
Availability Zone: us-east-1a (use1-az1) Zone type: Availability Zone
IP addresses available: 11 CIDR: 190.160.0.16/28)

Create new subnet

Auto-assign public IP Info

Disable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

- ○ Create security group
- ◉ Select existing security group

Common security groups Info

Select security groups

Compare security group rules

Security groups that you add or remove here will be added to or removed from all your network interfaces.

One Public and Private Instance launched

Instances (3) Info

Last updated less than a minute ago | Connect | Instance state | Actions | Launch instances

Saved filter sets: *Choose filter set*

Filter Instance by attribute or tag (case-sensitive)

1

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone
new-private-instance-vqr	i-08272b39beb87e5c1	Running	t3.micro	Initializing	us-east-1a
new-public-instance-vqr	i-04e943bb8b537e43c	Running	t3.micro	3/3 checks passed..	us-east-1a

7. Deploy Apache server on both EC2 instances with a sample index.html file.

Connect to the **Public EC2 Instance** using SSH. → Update the system packages. → Install Apache using sudo dnf install httpd -y. → Start Apache using sudo systemctl start httpd. → Enable Apache using sudo systemctl enable httpd. → Create a sample index.html file in /var/www/html/.→ Add the content: Welcome to Public EC2 Instance.→ Verify Apache status using sudo systemctl status httpd.
Connect to the **Private EC2 Instance** through the public instance (SSH). → Update the system packages. → Install Apache using sudo dnf install httpd -y. → Start Apache using sudo systemctl start httpd.→ Enable Apache using sudo systemctl enable httpd. → Create a sample index.html file in /var/www/html/. → Add the content: Welcome to Private EC2 Instance.
→ Verify Apache status using sudo systemctl status httpd. → Test both servers using curl http://localhost:80 . → Verify that both instances display their respective sample HTML pages.

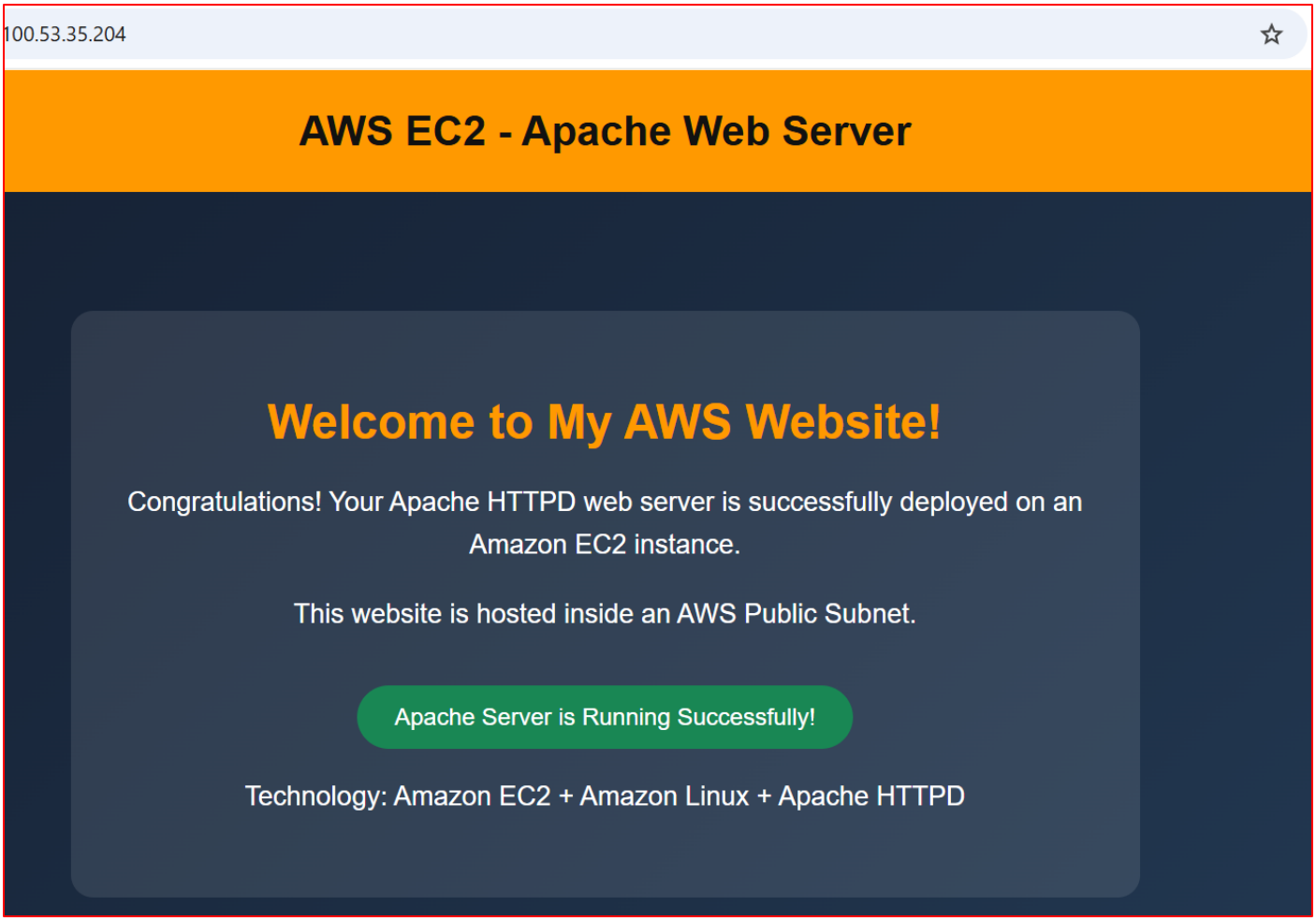
```
root@ip-190-160-0-7:~
[root@ip-190-160-0-7 ~]# yum install httpd
Last metadata expiration check: 0:01:58 ago on Thu Nov 12 12:19:22 2026.
Dependencies resolved.
================================================================================
 Package                    Architecture                Version
================================================================================
Installing:
 httpd                      x86_64                      2.4.69-1.amzn2023.0.2
Installing dependencies:
 apr                        x86_64                      1.7.5-1.amzn2023.0.4
 apr-util                   x86_64                      1.6.5-1.amzn2023.0.1
 apr-util-lmdb              x86_64                      1.6.5-1.amzn2023.0.1
 generic-logos-httpd        noarch                      18.0.0-12.amzn2023.0.3
 httpd-core                 x86_64                      2.4.69-1.amzn2023.0.2
 httpd-filesystem           noarch                      2.4.69-1.amzn2023.0.2
 httpd-tools                x86_64                      2.4.69-1.amzn2023.0.2
 libbrotli                  x86_64                      1.1.0-1.amzn2023.0.2
 mailcap                    noarch                      2.1.49-3.amzn2023.0.3
Installing weak dependencies:
 apr-util-openssl           x86_64                      1.6.5-1.amzn2023.0.1
 mod_http2                  x86_64                      1:2.4.69-1.amzn2023.0.2
 mod_lua                    x86_64                      2.4.69-1.amzn2023.0.2
```

```
Installed:
  apr-1.7.5-1.amzn2023.0.4.x86_64                  apr-util-1.6.5-1.amzn2023.0.1.x86_64
  apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64     generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
  httpd-core-2.4.69-1.amzn2023.0.2.x86_64          httpd-filesystem-2.4.69-1.amzn2023.0.2.noarch
  libbrotli-1.1.0-1.amzn2023.0.2.x86_64            mailcap-2.1.49-3.amzn2023.0.3.noarch
  mod_lua-2.4.69-1.amzn2023.0.2.x86_64

Complete!
[root@ip-190-160-0-7 ~]#
```

```
[root@ip-190-160-0-7 ~]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@ip-190-160-0-7 ~]# systemctl start httpd
[root@ip-190-160-0-7 ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: active (running) since Thu 2026-11-12 12:23:26 UTC; 9s ago
       Docs: man:httpd.service(8)
   Main PID: 26704 (httpd)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 177 (limit: 1059)
     Memory: 14.1M
        CPU: 67ms
     CGroup: /system.slice/httpd.service
             ├─26704 /usr/sbin/httpd -DFOREGROUND
             ├─26705 /usr/sbin/httpd -DFOREGROUND
             ├─26706 /usr/sbin/httpd -DFOREGROUND
             ├─26727 /usr/sbin/httpd -DFOREGROUND
             └─26728 /usr/sbin/httpd -DFOREGROUND

Nov 12 12:23:26 ip-190-160-0-7.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Nov 12 12:23:26 ip-190-160-0-7.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Nov 12 12:23:26 ip-190-160-0-7.ec2.internal httpd[26704]: Server configured, listening on: port 80
[root@ip-190-160-0-7 ~]#
```


100.53.35.204
AWS EC2 - Apache Web Server
Welcome to My AWS Website!
Congratulations! Your Apache HTTPD web server is successfully deployed on an Amazon EC2 instance.
This website is hosted inside an AWS Public Subnet.
Apache Server is Running Successfully!
Technology: Amazon EC2 + Amazon Linux + Apache HTTPD


ec2-user@ip-190-160-0-25:~
[root@ip-190-160-0-7 html]# ssh -i key.pem ec2-user@190.160.0.25
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
[ec2-user@ip-190-160-0-25 ~]$
Connected from Public
to private instance

root@ip-190-160-0-25:~

```
[root@ip-190-160-0-7 html]# ssh -i key.pem ec2-user@190.160.0.25
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-190-160-0-25 ~]$ sudo su -
[root@ip-190-160-0-25 ~]# yum update
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-190-160-0-25 ~]# yum upgrade
Last metadata expiration check: 0:00:09 ago on Thu Nov 12 13:15:36 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-190-160-0-25 ~]# yum install httpd -y
Last metadata expiration check: 0:00:24 ago on Thu Nov 12 13:15:36 2026.
Dependencies resolved.
```

```
[root@ip-190-160-0-25 ~]# curl localhost:80
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
[root@ip-190-160-0-25 ~]#
```

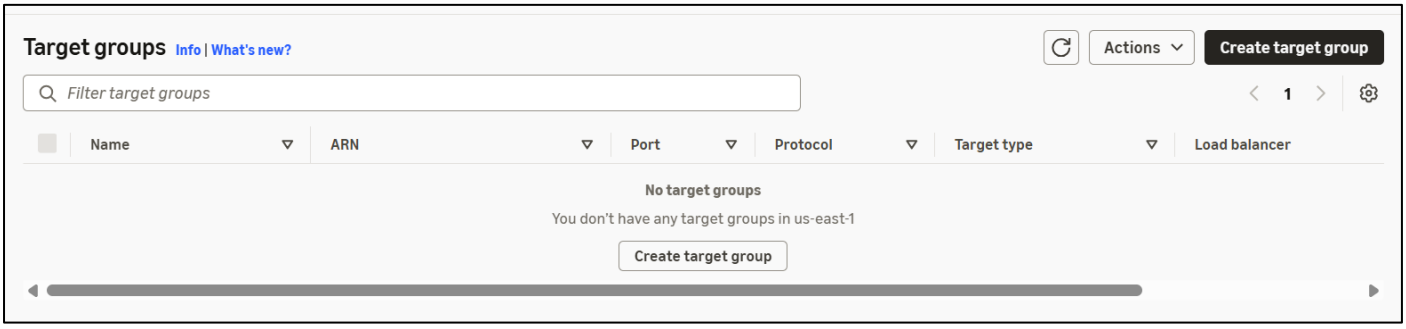
```
Complete!
[root@ip-190-160-0-25 ~]# systemctl update httpd
Unknown command verb update.
[root@ip-190-160-0-25 ~]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/http
[root@ip-190-160-0-25 ~]# systemctl start httpd
[root@ip-190-160-0-25 ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: active (running) since Thu 2026-11-12 13:17:16 UTC; 7s ago
       Docs: man:httpd.service(8)
   Main PID: 27865 (httpd)
     Status: "Started, listening on: port 80"
      Tasks: 177 (limit: 1016)
     Memory: 13.7M
        CPU: 70ms
     CGroup: /system.slice/httpd.service
             ├─27865 /usr/sbin/httpd -DFOREGROUND
             ├─27866 /usr/sbin/httpd -DFOREGROUND
             ├─27867 /usr/sbin/httpd -DFOREGROUND
             ├─27868 /usr/sbin/httpd -DFOREGROUND
             └─27869 /usr/sbin/httpd -DFOREGROUND

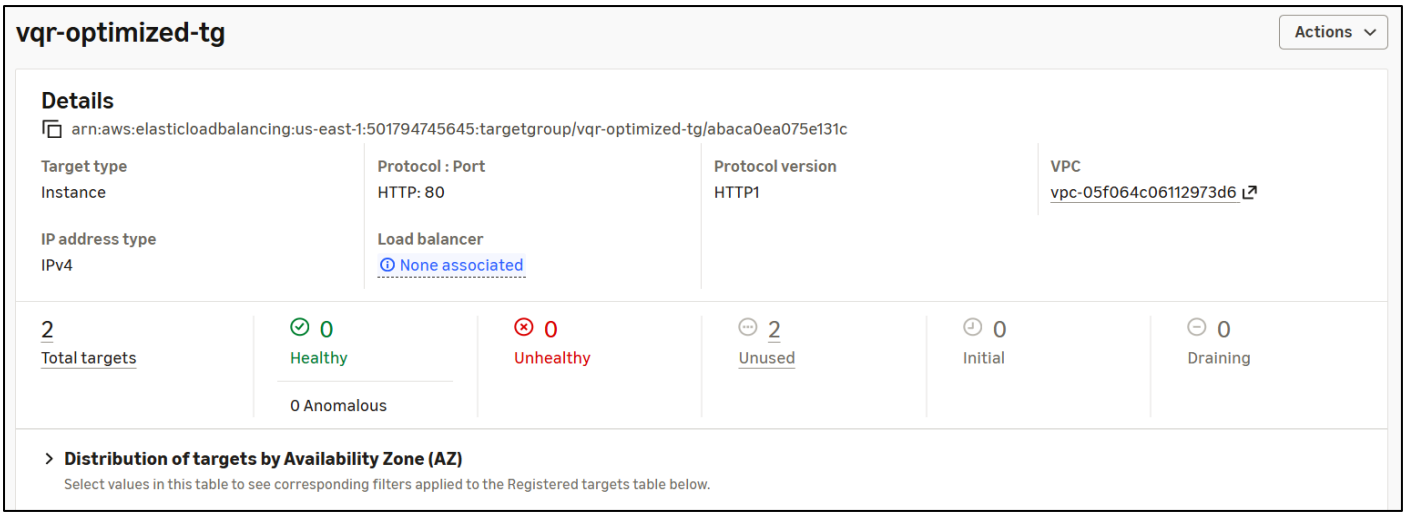
Nov 12 13:17:16 ip-190-160-0-25.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server
Nov 12 13:17:16 ip-190-160-0-25.ec2.internal httpd[27865]: Server configured, listening on: port 80
Nov 12 13:17:16 ip-190-160-0-25.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server
[root@ip-190-160-0-25 ~]#
```

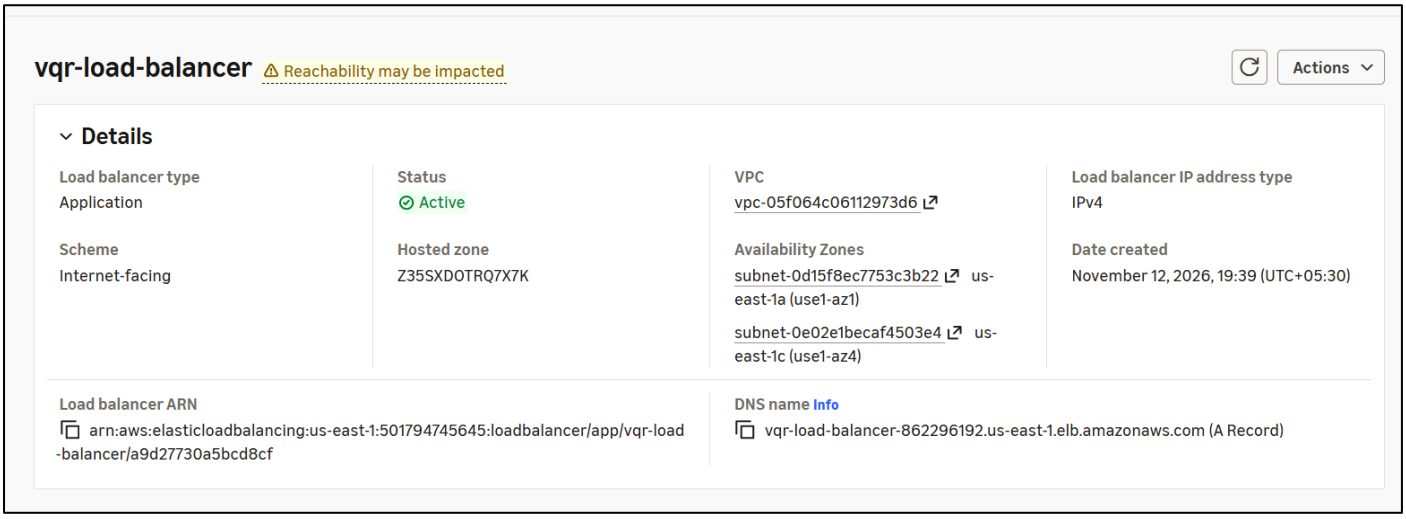
Private

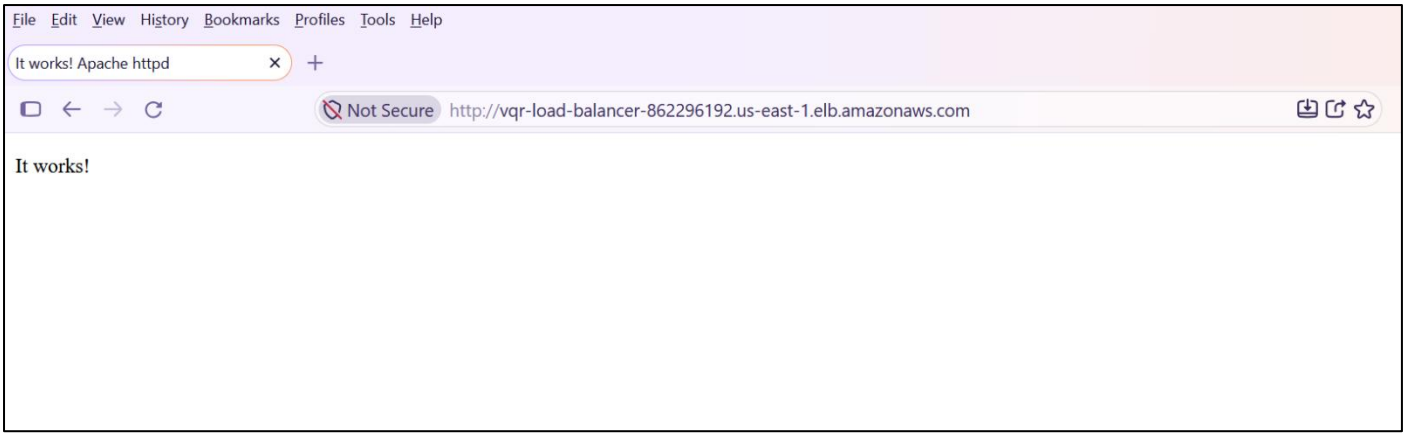
8. Create one application load balancer and attach it to both EC2 instances.

Search for **EC2** and open the EC2 Dashboard. → Select **Load Balancers** from the left navigation pane. → Click **Create load balancer**. → Select **Application Load Balancer**. → Enter name: My-ALB.
→ Select scheme: **Internet-facing**. → Select IP address type: **IPv4**. → Select VPC: My-VPC. → Select both Availability Zones and their public subnets. → Select or create a security group allowing HTTP traffic on port 80. → Under Listeners and routing, select **HTTP : 80**.→ Click **Create target group**.
→ Select target type: **Instances**. → Enter target group name: My-TG. → Select protocol: **HTTP** and port: 80. → Select VPC: My-VPC. → Configure health check path: /var/www/html/index.html. → Click **Next**. → Select both Public-Instance and Private-Instance. → Click **Include as pending below**.
→ Click **Create target group**.
Return to the load balancer configuration. → Select My-TG as the default target group. → Click **Create load balancer**. → Wait until the load balancer state becomes **Active**. → Verify that both EC2 instances are registered in the target group. → Ensure both instances show **Healthy** status.
→ Copy the ALB DNS name and open it in a web browser to test.



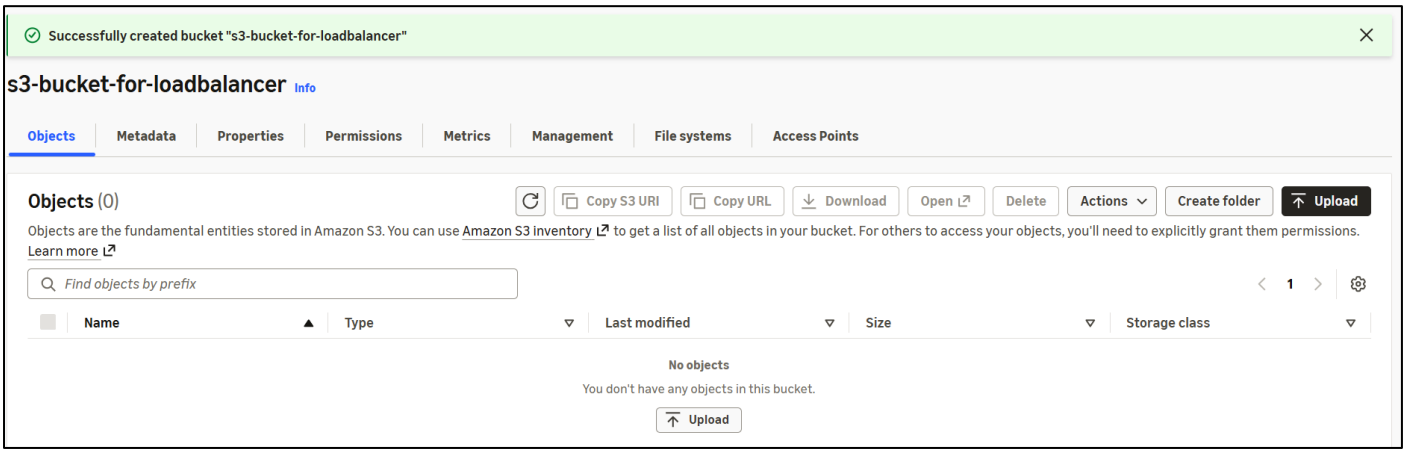


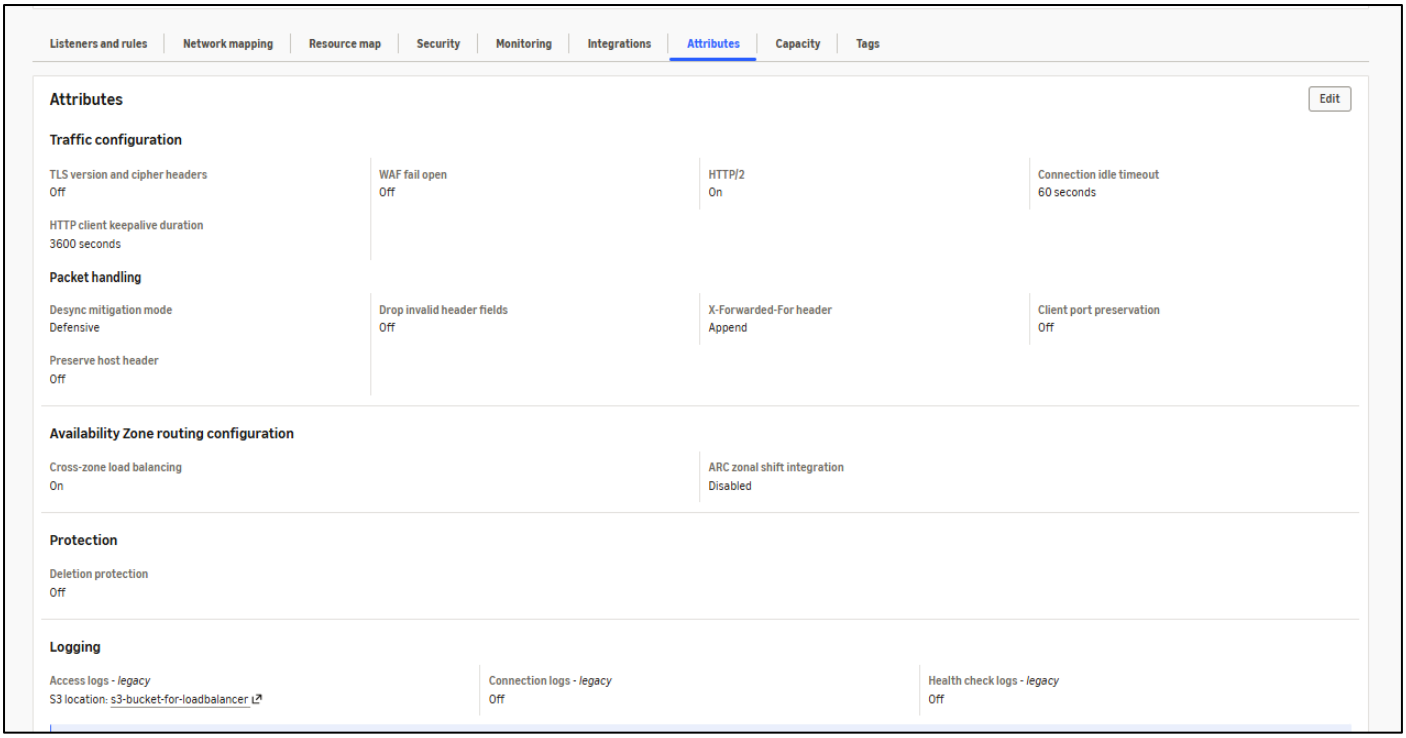


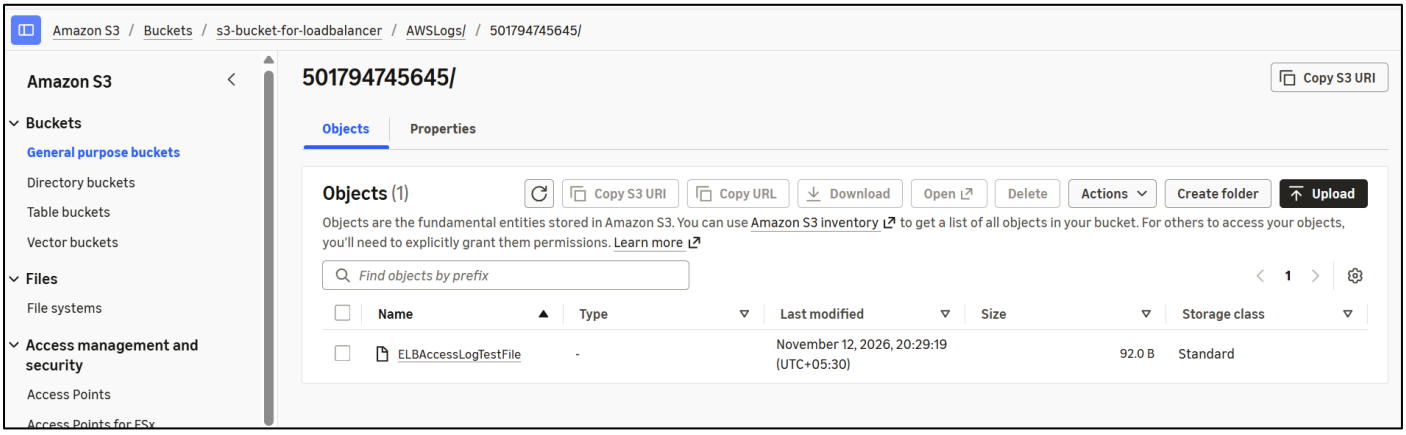


9. Store application load balancer logs in S3.

Click **Create bucket** → Enter bucket name: my-alb-logs-bucket. → Select region: **US East (N. Virginia) – us-east-1** → Keep the default settings. → Click **Create bucket**. → Open the created S3 bucket. → Go to the **Permissions** tab. → Edit the **Bucket policy** to allow the Elastic Load Balancing service to deliver access logs. → Save the bucket policy. → Open the **EC2 Dashboard**. → Select **Load Balancers**. → Select your Application Load Balancer (My-ALB). → Open the **Attributes** tab.
→ Click **Edit**. → Enable **Access logs**. → Enter the S3 bucket name: my-alb-logs-bucket. → Enter an optional prefix: alb-logs. → Click **Save changes**.
Verify that access logging is enabled. → Generate some traffic through the ALB DNS name.
→ Open the S3 bucket and check the alb-logs folder for log files.→ Verify that ALB access logs are successfully stored in S3.

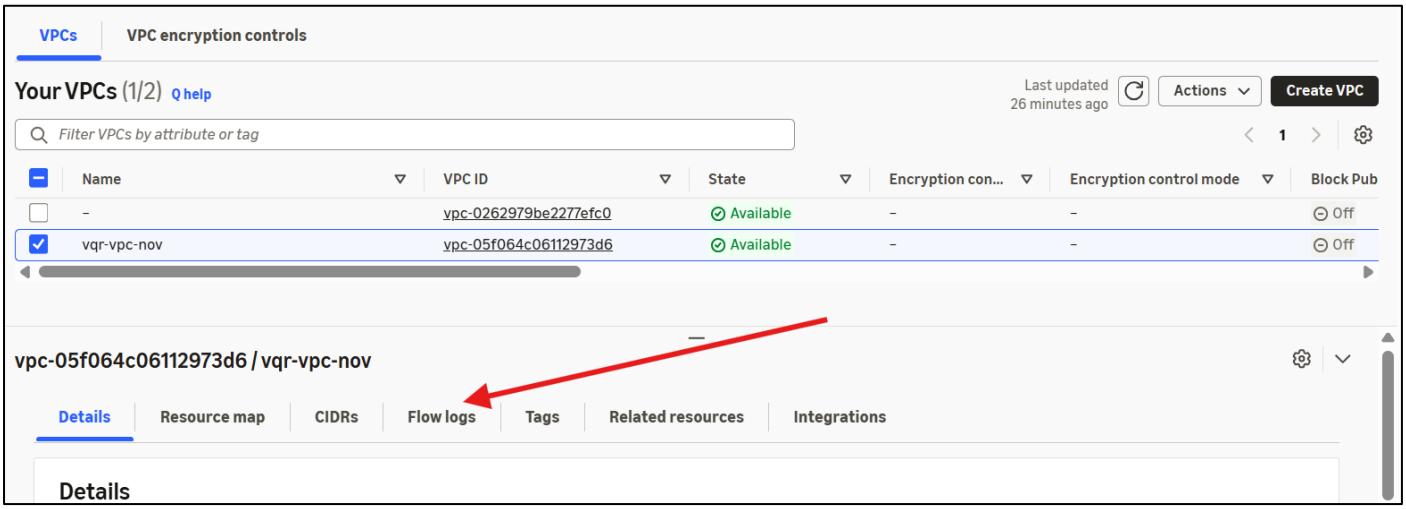


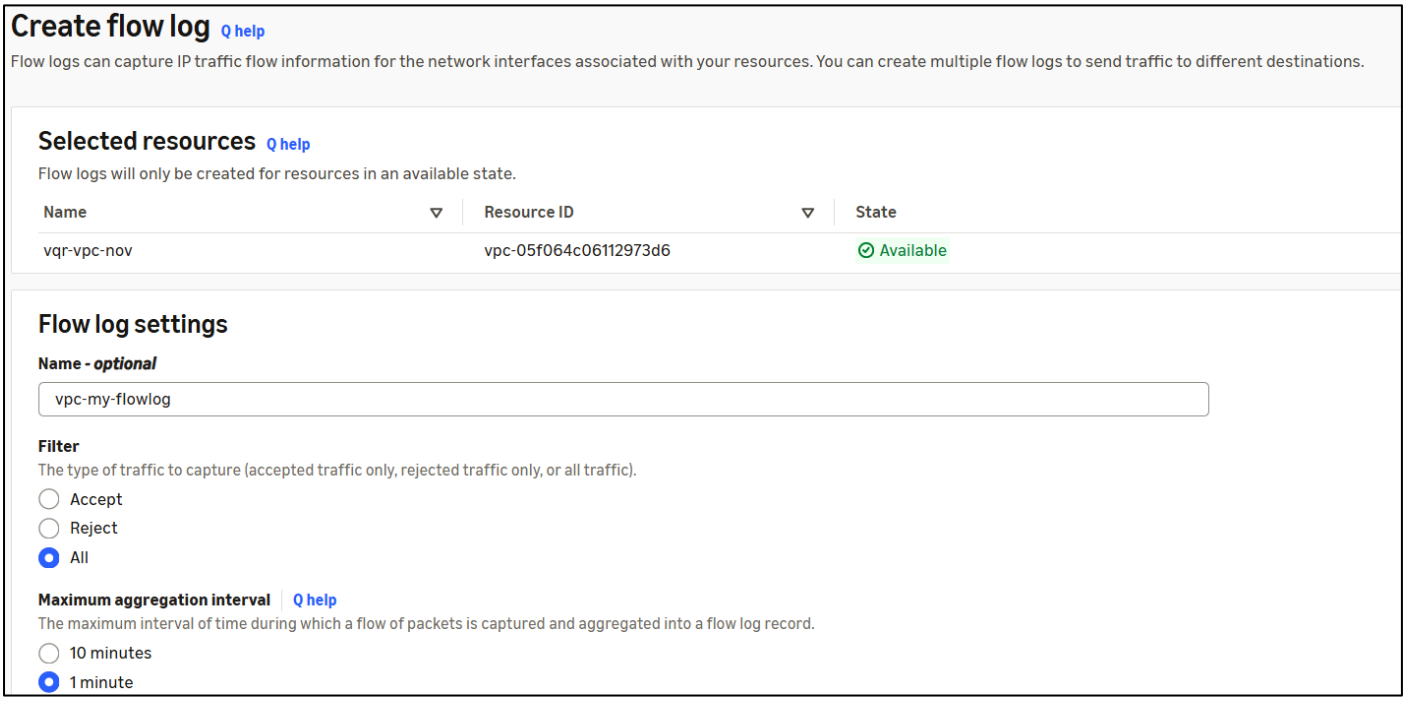

Listeners and rules
Network mapping
Resource map
Security
Monitoring
Integrations
Attributes
Capacity
Tags
Attributes
Edit
Traffic configuration
TLS version and cipher headers
Off
WAF fail open
Off
HTTP/2
On
Connection idle timeout
60 seconds
HTTP client keepalive duration
3600 seconds
Packet handling
Desync mitigation mode
Defensive
Drop invalid header fields
Off
X-Forwarded-For header
Append
Client port preservation
Off
Preserve host header
Off
Availability Zone routing configuration
Cross-zone load balancing
On
ARC zonal shift integration
Disabled
Protection
Deletion protection
Off
Logging
Access logs - legacy
S3 location: s3-bucket-for-loadbalancer
Connection logs - legacy
Off
Health check logs - legacy
Off


Amazon S3 / Buckets / s3-bucket-for-loadbalancer / AWSLogs/ / 501794745645/
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems
Access management and security
Access Points
Access Points for FSx
501794745645/
Copy S3 URI
Objects
Properties
Objects (1)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
1
Name
Type
Last modified
Size
Storage class
ELBAccessLogTestFile
-
November 12, 2026, 20:29:19 (UTC+05:30)
92.0 B
Standard

10.Store the VPC flow logs in a CloudWatch log group.

Open the **AWS Management Console**. → Search for **VPC** and open the VPC Dashboard. → Select **Your VPCs** from the left navigation pane. → Select your VPC (My-VPC). → Click the **Flow logs** tab. → Click **Create flow log**. → Enter name: My-VPC-Flow-Logs. → Select filter: **All**. → Select maximum aggregation interval: **1 minute**. → Under Destination, select **Send to CloudWatch Logs**. → Create a new CloudWatch log group named My-VPC-Log-Group.
Select an IAM role that allows VPC Flow Logs to publish data to CloudWatch Logs. → Select the log format: **AWS default format**. → Click **Create flow log**. → Open the **CloudWatch Dashboard**. → Select **Logs** → **Log groups**. → Open My-VPC-Log-Group.





Destination

The destination to which to publish the flow log data.

- (•) Send to CloudWatch Logs
- ( ) Send to an Amazon S3 bucket
- ( ) Send to Amazon Data Firehose in the same account
- ( ) Send to Amazon Data Firehose in a different account

Destination log group | Q help

The name of an existing log group or the name of a new log group that will be created when you create this flow log. A new log stream is created for each monitored network interface.

Choose or create a log group name...

Service access

VPC flow logs require permissions to create log groups and publish events in CloudWatch.

- (•) Use an existing service role
- ( ) Create and use a new service role

Successfully created flow log for the following resource:

- vpc-05f064c06112973d6

fl-05baacc6eaf44775f / vpc-my-flowlog

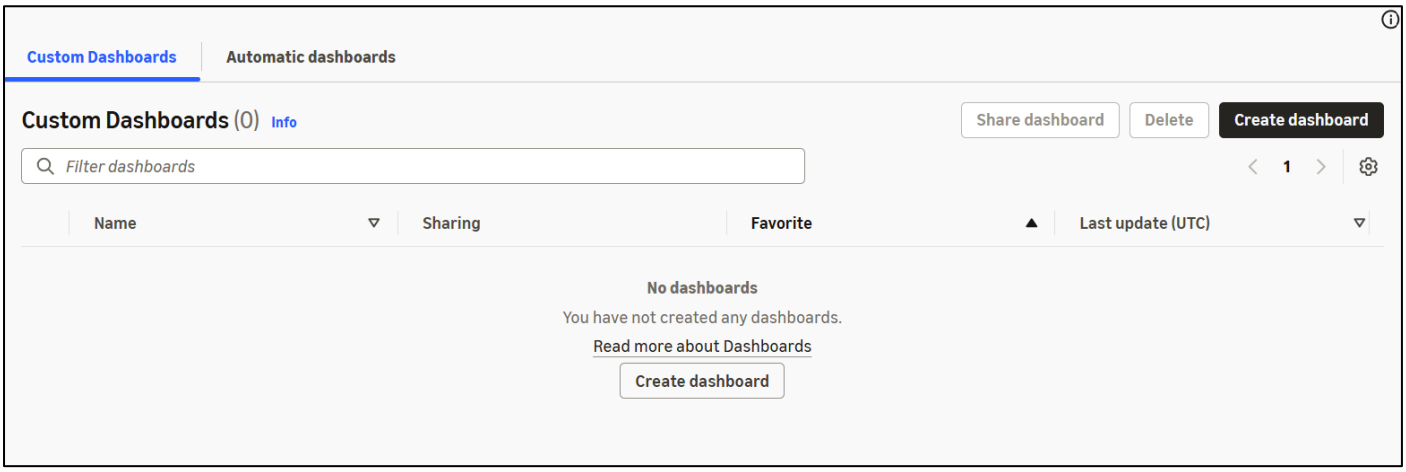
Actions

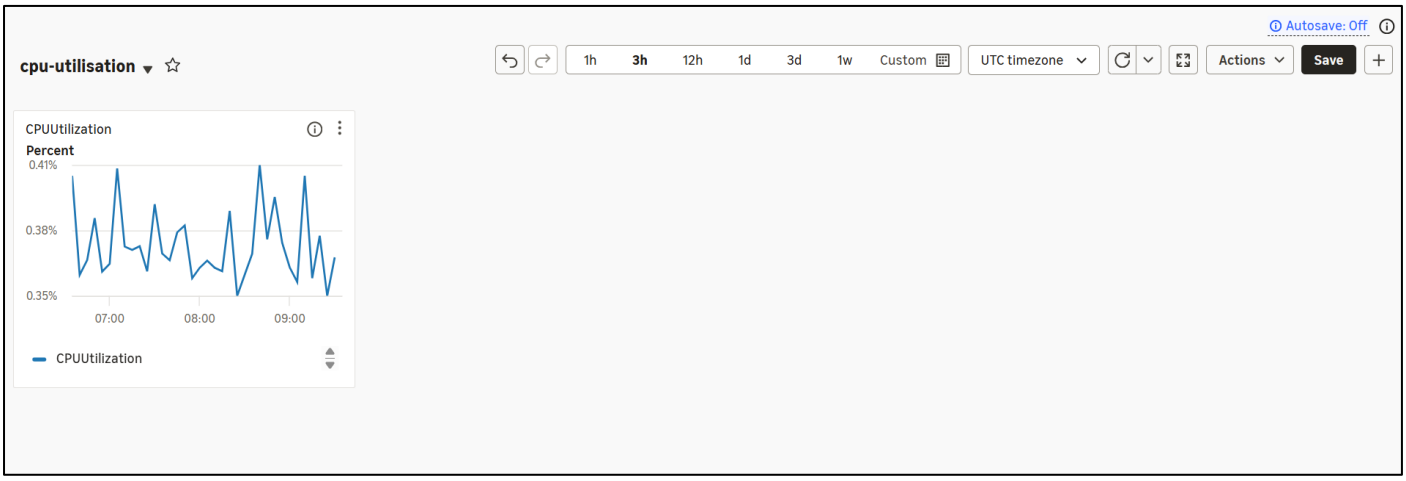
Details

Flow Log ID fl-05baacc6eaf44775f	Destination Type cloud-watch-logs	Traffic Type All	File Format -
Name vpc-my-flowlog	Destination Name aws-cloudtrail-logs-501794745645-59048cdd	Max Aggregation Interval 1 minute	Hive Compatible Partitions -
State Active	IAM Role arn:aws:iam::501794745645:role/service-role/VPCFlowLogs-Cloudwatch-1794493144480	Log Format Default	Partition Logs -
Creation Time Thursday, November 12, 2026 at 19:50:48 GMT+5:30	Cross Account IAM Role -	Tag field specifications -	

11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

Open the **AWS Management Console**. → Search for **CloudWatch** and open the CloudWatch Dashboard. **Create CPU Utilization Dashboard** → Select **Dashboards** from the left navigation pane. → Click **Create dashboard**. → Enter dashboard name: EC2-Monitoring-Dashboard. → Click **Create dashboard**. → Select **Line** as the widget type. → Select **Metrics**. → Choose **EC2** → **Per-Instance Metrics**. → Select both EC2 instances. → Select the **CPUUtilization** metric. → Click **Create widget**. → Set the time range to **Last 1 hour**. → Click **Save dashboard**.
Monitor Apache Service → Connect to both EC2 instances using SSH. → Install the CloudWatch Agent on both instances. → Configure the CloudWatch Agent to collect Apache service status using a custom monitoring script or CloudWatch Agent-supported metrics. → Configure the agent to publish the Apache status metric to CloudWatch. → Start and enable the CloudWatch Agent. → Open **CloudWatch** → **Metrics**. → Select the custom metric namespace containing the Apache service status.





```
# AWS CLI command
AWS_CLI_CMD="aws cloudwatch put-metric-data"

# Check if Apache (httpd) is running
if pgrep -x httpd > /dev/null 2>&1; then
    STATUS=1
else
    STATUS=0
fi

# Send the metric to CloudWatch
$AWS_CLI_CMD \
    --namespace "$NAMESPACE" \
    --metric-name "$METRIC_NAME" \
    --value "$STATUS" \
    --unit Count \
    --region "$REGION"

# Log the status
echo "$(date '+%Y-%m-%d %H:%M:%S') ApacheStatus: $STATUS" >> /var/log/apache_monitor.log
[root@ip-190-160-0-7 ~]# ./httpd.sh

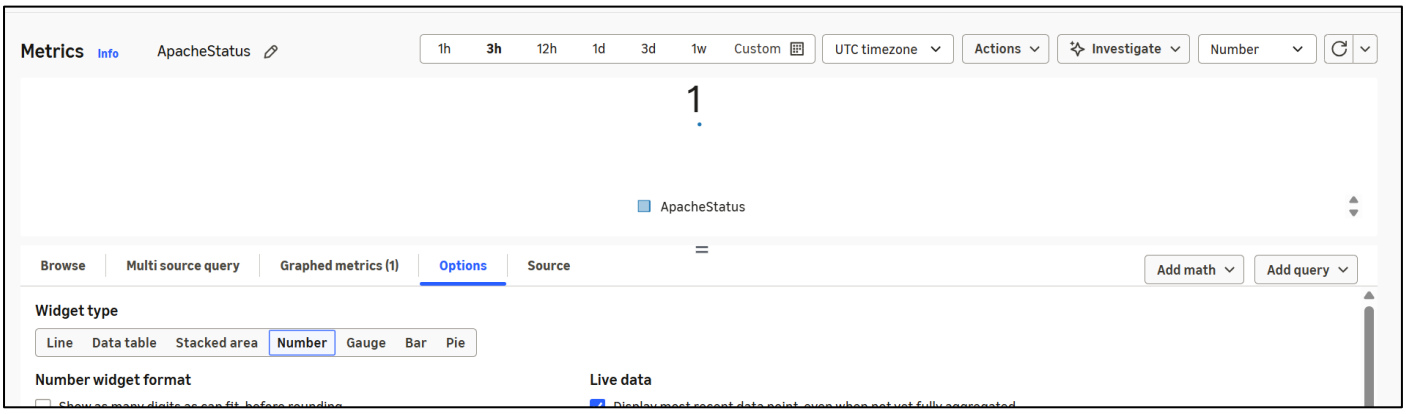
aws: [ERROR]: An error occurred (NoCredentials): Unable to locate credentials. You can configure creden
[root@ip-190-160-0-7 ~]# aws login
Attempting to open your default browser. If the browser does not open, open the following URL.
If you are unable to open the URL on this device, run this command again with the '--remote' option.

https://us-east-1.signin.aws.amazon.com/v1/authorize?response_type=code&client_id=arn%3Aaws%3Asignin%3A
f9&code_challenge_method=SHA-256&scope=openid&redirect_uri=http%3A%2F%2F127.0.0.1%3A42893%2Foauth%2Fcal

aws configure
^C
[root@ip-190-160-0-7 ~]# aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the

AWS Access Key ID [None]: AKIAXJVKMFUW6FSKCIEJ
AWS Secret Access Key [None]: GE7uwBqWVMuhp/a/5BxgeuUHn1l86B7vOctKTMh7
Default region name [None]:
Default output format [None]: [root@ip-190-160-0-7 ~]#
[root@ip-190-160-0-7 ~]# ls
httpd.sh
[root@ip-190-160-0-7 ~]# ./httpd.sh
[root@ip-190-160-0-7 ~]#
```



12. If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

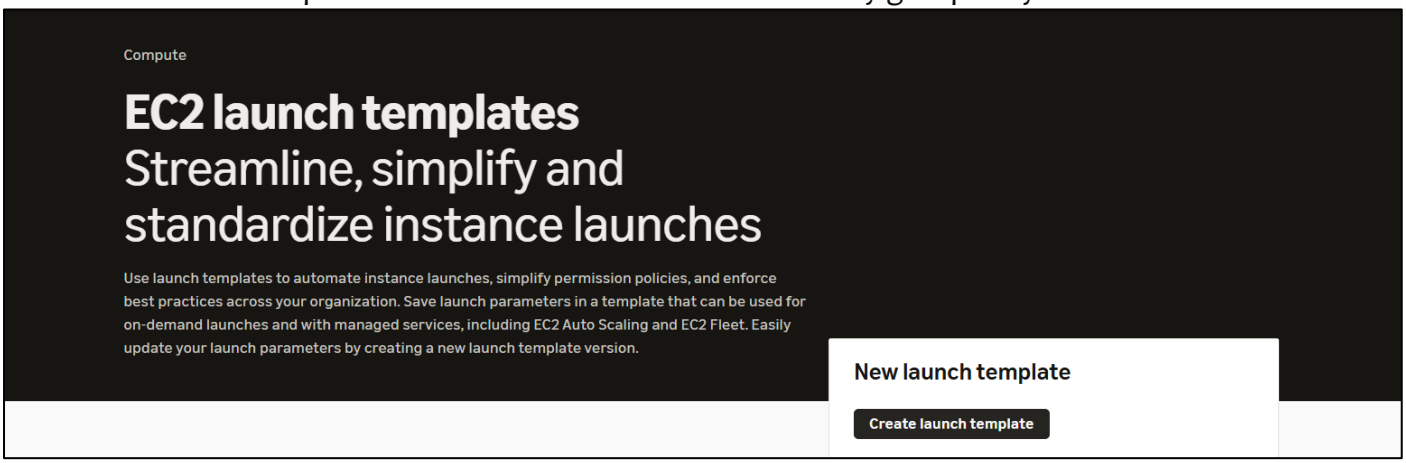
Select **Launch Templates** from the left navigation pane. → Click **Create launch template**. → Enter name: My-Launch-Template. → Select the Amazon Linux 2023 AMI. → Select instance type: t2.micro (if available). → Select the required key pair. → Configure the security group to allow HTTP traffic on port 80. → Select **My-VPC** as the VPC. → Add user data to install and start Apache automatically.→ Click **Create launch template**.

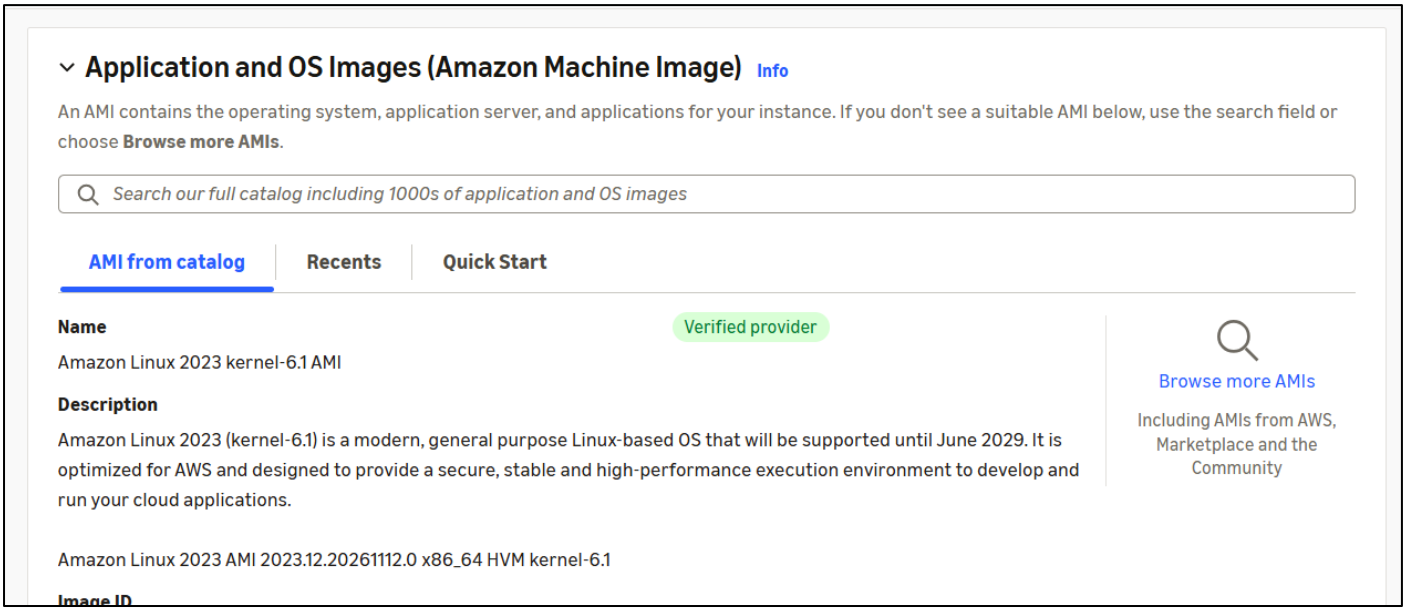
Create Auto Scaling Group → Select **Auto Scaling Groups** from the left navigation pane. → Click **Create Auto Scaling group**. → Enter name: My-ASG. → Select launch template: My-Launch-Template. → Select VPC: My-VPC. → Select the required subnets. → Attach the existing Application Load Balancer target group (My-TG). → Set desired capacity: 2. → Set minimum capacity: 1. → Set maximum capacity: 5. → Click **Create Auto Scaling group**.

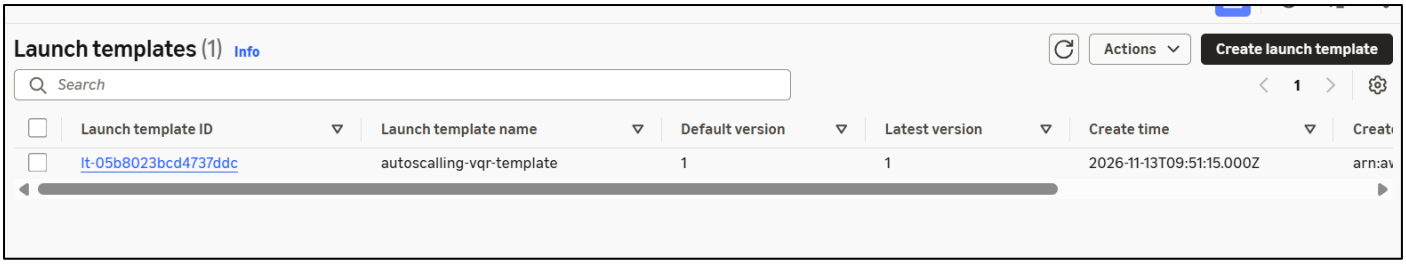
Configure CPU-Based Scaling Policy → Open the Auto Scaling Groups page. → Select My-ASG. → Go to the **Automatic scaling** tab. → Click **Create dynamic scaling policy**. → Enter policy name: CPU-Scaling-Policy. → Select **Target tracking scaling policy**. → Select metric: **Average CPU Utilization**. → Set target value: 70. → Set instance warm-up time: 300 seconds.→ Click **Create**.

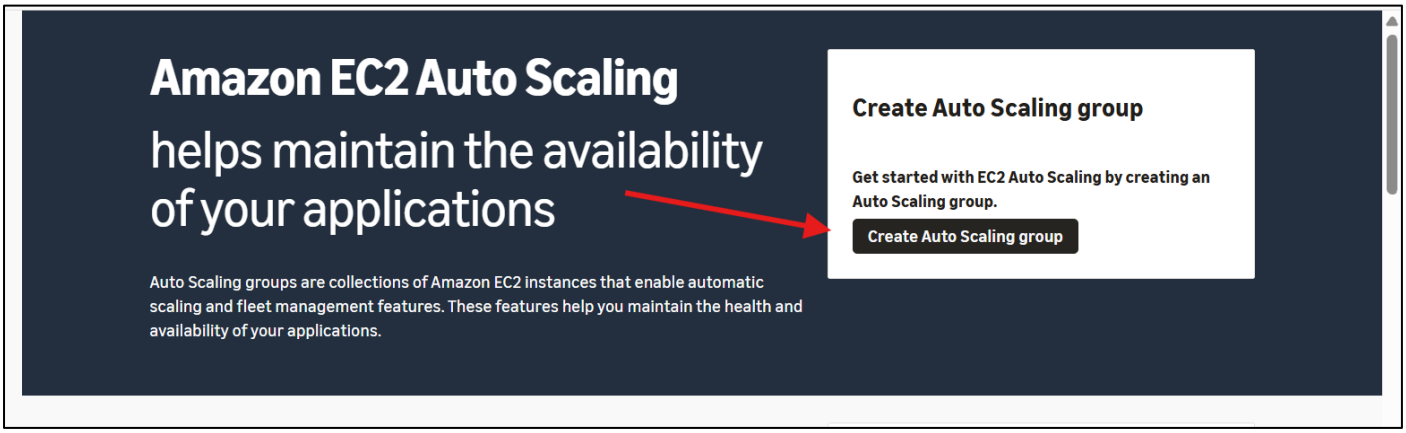
Verify Auto Scaling → Open **CloudWatch** → **Alarms**. → Verify that the CPU-based scaling alarm is created. → Generate CPU load on the EC2 instances for testing. → Verify that CPU utilization triggers the scaling policy. → Check the Auto Scaling Group's **Activity** tab. → Verify that a new EC2 instance is launched automatically when required. → Confirm that the new instance is registered with the ALB target group.

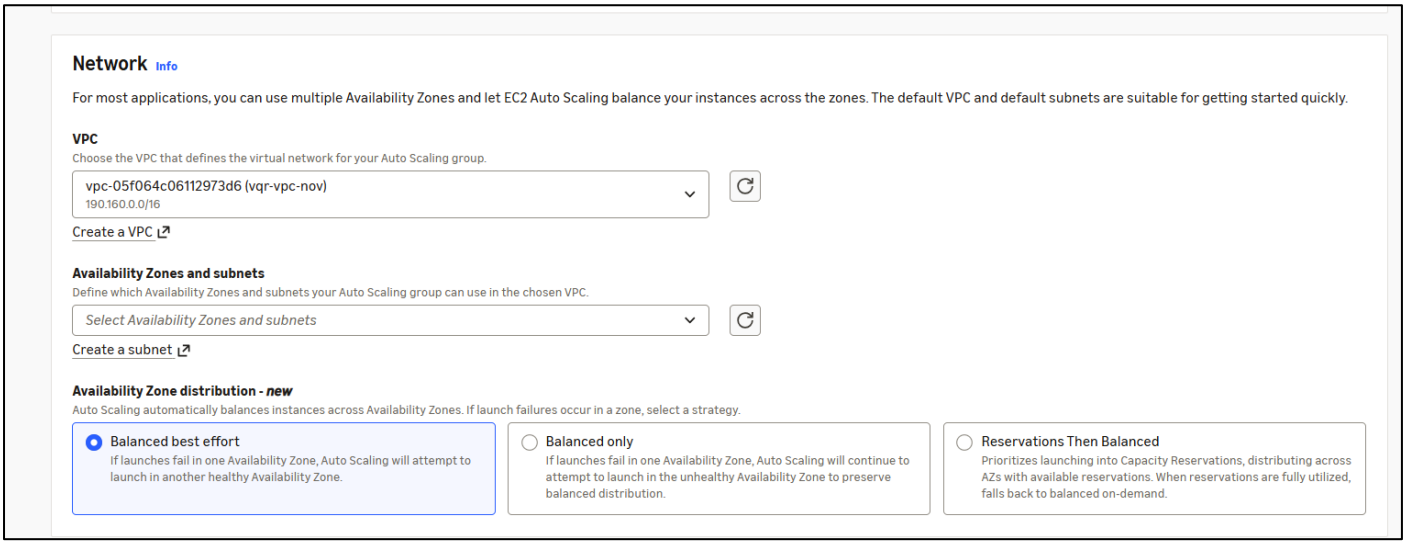
Note: Target tracking maintains CPU utilization around 70%; it may scale out before CPU exceeds 70%. Ensure the launch template uses the correct subnets and security group for your ALB architecture

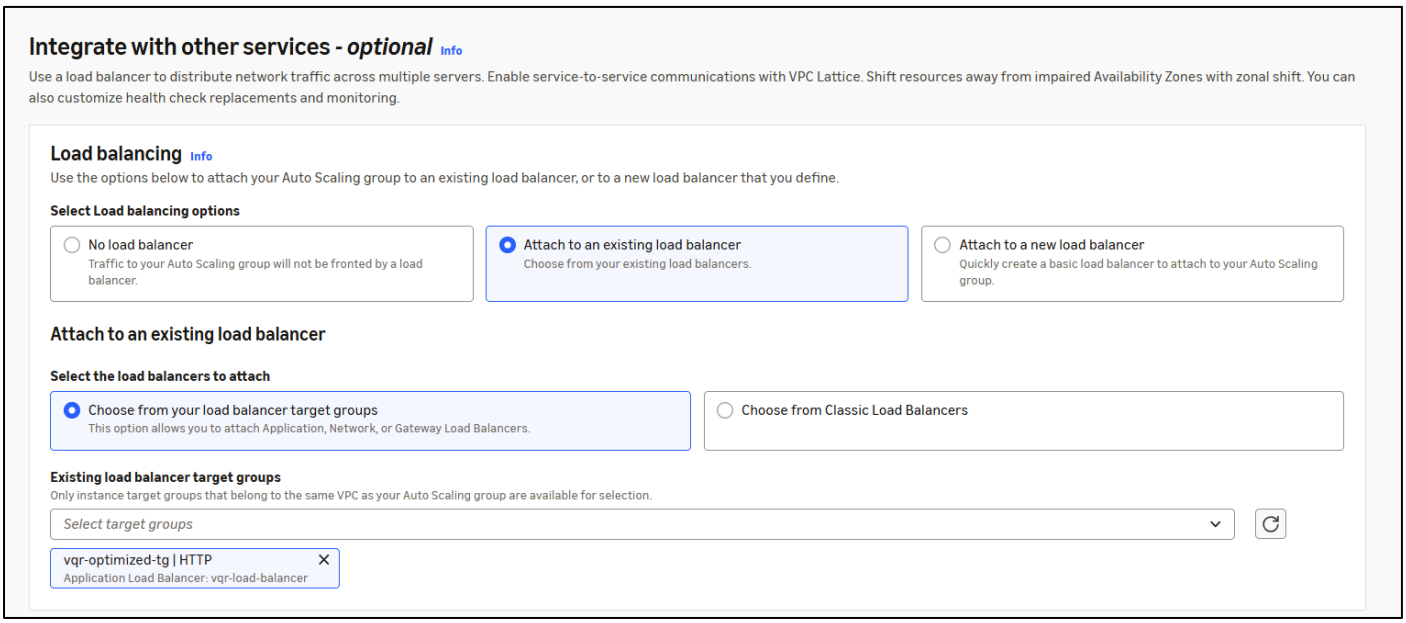


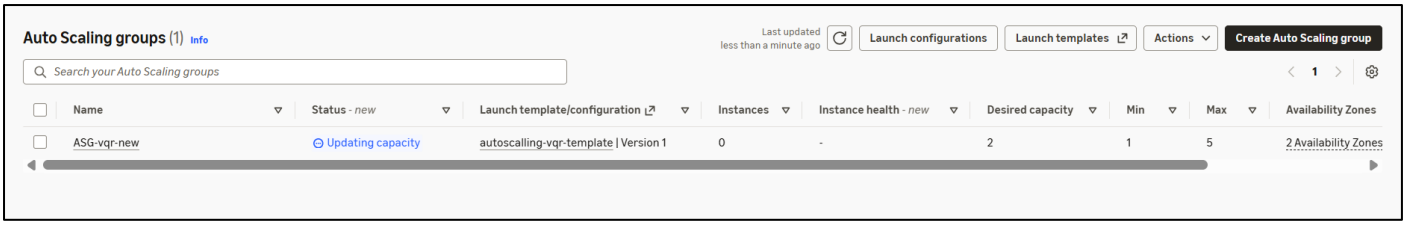


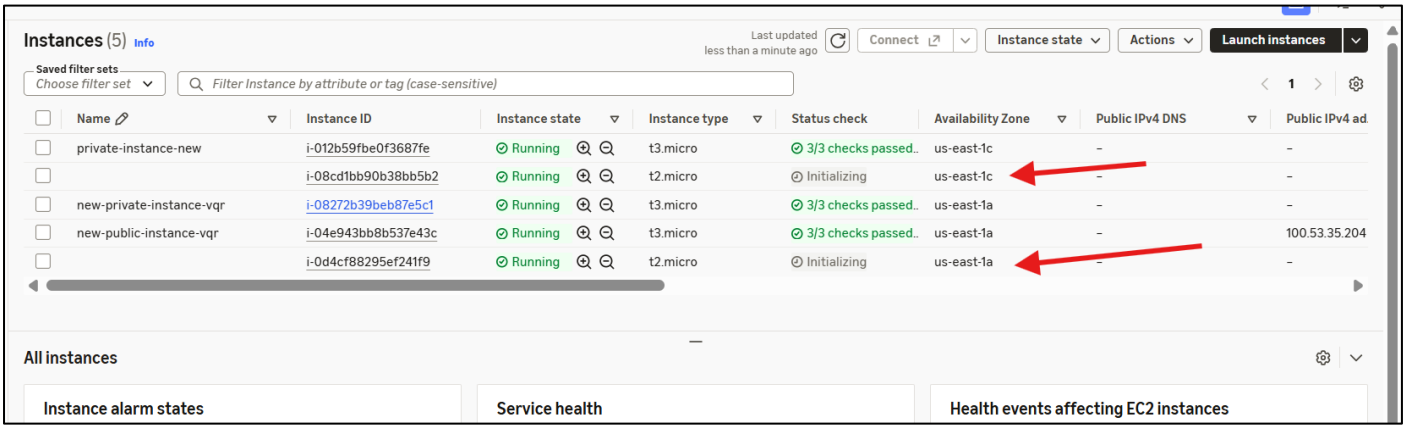

Launch templates (1) Info
Actions
Create launch template
Search
1
Launch template ID
Launch template name
Default version
Latest version
Create time
lt-05b8023bcd4737ddc
autoscalling-vqr-template
1
1
2026-11-13T09:51:15.000Z


Amazon EC2 Auto Scaling
helps maintain the availability of your applications
Auto Scaling groups are collections of Amazon EC2 instances that enable automatic scaling and fleet management features. These features help you maintain the health and availability of your applications.
Create Auto Scaling group
Get started with EC2 Auto Scaling by creating an Auto Scaling group.
Create Auto Scaling group


Network Info
For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.
VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.
vpc-05f064c06112973d6 (vqr-vpc-nov)
190.160.0.0/16
Create a VPC
Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.
Select Availability Zones and subnets
Create a subnet
Availability Zone distribution - new
Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.
Balanced best effort
If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.
Balanced only
If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.
Reservations Then Balanced
Prioritizes launching into Capacity Reservations, distributing across AZs with available reservations. When reservations are fully utilized, falls back to balanced on-demand.


Integrate with other services - optional Info
Use a load balancer to distribute network traffic across multiple servers. Enable service-to-service communications with VPC Lattice. Shift resources away from impaired Availability Zones with zonal shift. You can also customize health check replacements and monitoring.
Load balancing Info
Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.
Select Load balancing options
No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.
Attach to an existing load balancer
Choose from your existing load balancers.
Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.
Attach to an existing load balancer
Select the load balancers to attach
Choose from your load balancer target groups
This option allows you to attach Application, Network, or Gateway Load Balancers.
Choose from Classic Load Balancers
Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.
Select target groups
vqr-optimized-tg | HTTP
Application Load Balancer: vqr-load-balancer


Auto Scaling groups (1) Info
Last updated less than a minute ago
Launch configurations
Launch templates
Actions
Create Auto Scaling group
Search your Auto Scaling groups
Name
Status - new
Launch template/configuration
Instances
Instance health - new
Desired capacity
Min
Max
Availability Zones
ASG-vqr-new
Updating capacity
autoscalling-vqr-template | Version 1
0
-
2
1
5
2 Availability Zones


Instances (5) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Public IPv4 ad
private-instance-new
i-012b59fbe0f3687fe
Running
t3.micro
3/3 checks passed..
us-east-1c
-
-
i-08cd1bb90b38bb5b2
Running
t2.micro
Initializing
us-east-1c
-
-
new-private-instance-vqr
i-08272b39beb87e5c1
Running
t3.micro
3/3 checks passed..
us-east-1a
-
-
new-public-instance-vqr
i-04e943bb8b537e43c
Running
t3.micro
3/3 checks passed..
us-east-1a
-
100.53.35.204
i-0d4cf88295ef241f9
Running
t2.micro
Initializing
us-east-1a
-
-
All instances
Instance alarm states
Service health
Health events affecting EC2 instances